Brooke Wheeless
April 18, 2026
CS 230

Draw It or Lose It

CS 230 Project Software Design Template

Version 1.0

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 4
Domain Model 4
Evaluation 6
Recommendations 7

Document Revision History

Version	Date	Author	Comments
1.0	03/22/26	Brooke Wheeless	Initial creation of software design document for The Gaming Room application.

Executive Summary

The client, The Gaming Room, would like to redevelop and expand its current game *Draw It or Lose It*, from an Android only app to a web-based game that serves multiple platforms. This requires a new software design that supports a distributed access environment that still meets the original requirements of the game. Client business requirements include supporting multiple teams and players, while maintaining unique game, team, and player names. Client technical requirements include a web-based design that only allows one instance of the game to exist at any given time. The proposed solution is to redesign *Draw It or Lose It* as a web-based application that can be accessed across multiple platforms. This approach meets business and technical needs by improving accessibility and supporting a design to manage unique games, teams, and players.

Requirements

The Gaming Room wants their game *Draw It or Lose It* redesigned as a web-based application that can support multiple platforms. The software design will be guided by the business and technical requirements.

Business Requirements:

1. A game must support one or more teams.
2. Each team must contain multiple players.
3. Game and team names must be unique.

Technical Requirements:

1. The application must support a web-based, distributed environment.
2. Only one instance of the game service may exist at any given time.
3. Unique identifiers must be used for each game, team, and player.
4. The software design must allow the game to be used across multiple platforms.
5. The system must manage game data efficiently.

Design Constraints

The design for *Draw It or Lose It* must support The Gaming Room's move from an Android-only application to a web-based application that works across multiple platforms. This creates limits that affect the platform, system design, and project.

Platform Constraints:

The current version of the game is limited to Android. The new design must support the use of multiple operating systems. The application should be accessible through a web-based approach instead of relying on one mobile platform.

Technical Constraints:

The application must function in a distributed environment to allow access through a web-based approach. It must also maintain unique game, team, and player names while allowing only one instance to exist at a time.

Business/Project Constraints:

The new design must meet the original requirements and needs of the current Android *Draw It or Lost It* game. The design should be easy to understand and reduce complexity. The development team should choose technology that is efficient and supports future growth without a new redesign.

System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

This diagram is a simple map of how the game is organized. It shows the main parts of the program and how they connect so the system can keep track of games, teams, and players. It is similar to how a sport league works, where a league contains games, each game has teams, and each team has players.

→ Design of Game:

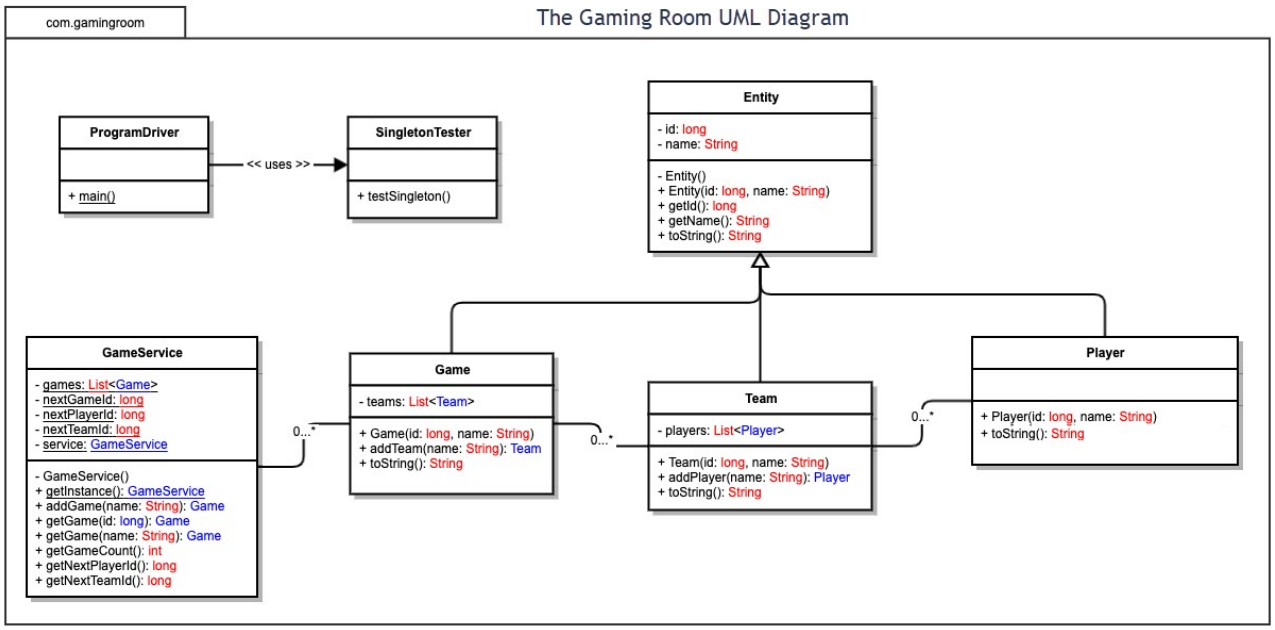
The Gaming Room UML Diagram shows the classes ProgramDriver, SingletonDriver, Entity, GameService, Game, Team, and Player, which are the main design of the game *Draw It or Lose It.* The GameService class is a central class that manages the collection of game objects. It keeps track of the next unique id for the games, teams, and player objects. The Game class represents a game and is associated with Team objects and the Team class is associated with Player objects. The result is a structure in which each game contains teams, which contains players.

→ Object-Oriented Principles:

Several core object-oriented programming principles are demonstrated in the diagram. The Entity class serves as a parent class for Game, Team, and player. As a result, inheritance, which is the OOP principle that allows a class to take on shared attributes and behavior from another class, is shown through the common attributes in these classes, such as id and name. Encapsulation is demonstrated in the diagram through the use of private attributes, which helps protect and control class data. The OOP abstraction, which represents general shared characteristics, is shown through the Entity class in the diagram by defining common features used by Game, Team, and Player classes.

→ How Application Works:

The application begins with the main driver class, ProgramDriver, which acts as the starting point of the program. ProgramDriver then uses SingletonTester to confirm the singleton pattern in GameService is working. This is to ensure only one instance exists at any given time. GameService manages creating and storing Game objects. Each Game object contains multiple Team objects, which contain multiple Plauer objects. As shown in the diagram, this creates a structure in which the main class is accessed and then builds data by utilizing game, team, and player objects.


com.gamingroom
The Gaming Room UML Diagram
ProgramDriver
+ main()
<< uses >>
SingletonTester
+ testSingleton()
Entity
- id: long
- name: String
- Entity()
+ Entity(id: long, name: String)
+ getId(): long
+ getName(): String
+ toString(): String
GameService
- games: List<Game>
- nextGameId: long
- nextPlayerId: long
- nextTeamId: long
- service: GameService
- GameService()
+ getInstance(): GameService
+ addGame(name: String): Game
+ getGame(id: long): Game
+ getGame(name: String): Game
+ getGameCount(): int
+ getNextPlayerId(): long
+ getNextTeamId(): long
0...*
Game
- teams: List<Team>
+ Game(id: long, name: String)
+ addTeam(name: String): Team
+ toString(): String
0...*
Team
- players: List<Player>
+ Team(id: long, name: String)
+ addPlayer(name: String): Player
+ toString(): String
0...*
Player
+ Player(id: long, name: String)
+ toString(): String

Evaluation

Comparison of operating systems for application development.

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side compatibility security performance scalability	Supports web servers, not typically used large-scale. Strong built in Unix-based security. Stable performance, not optimized for high-traffic environment. Limited in scaling.	Supports web hosting. Widely used for secure sever environments. High performance and stability. Highly scalable and commonly used in enterprise or cloud systems.	Good support with Microsoft based applications. Strong security that requires frequent maintenance. Reliable performance, needs more system resources.	Not designed to host web applications. Security not suitable for server environment. Performance limited to processing power. Not scalable.
Client Side development → cost → time expertise user access	Moderate cost, requires Mac hardware. Moderate time, few compatibility issues. Apple-specific knowledge needed. Small userbase.	Low cost of development due to open-source and free tools. Longer development time due to variations. More technical knowledge needed. Small userbase.	Moderate cost, paid licenses and tools necessary. Fast development time. Widely supported. Large user base due to high compatibility.	High cost and longer development time, extra testing required. Specific expertise and tools needed. User access very high since mobile devices are widely used.
Development Tools languages IDEs tools	Java, JavaScript, Python, Visual Studio Code, Eclipse. Strong tools for development for iOS and web applications.	Java, JavaScript, Python, C++, Visual Studio Code, Eclipse. IntelliJ IDEA. Mainly used for server-side and web applications.	Java, C#, JavaScript, Python, Visual Studio, Visual Studio Code, Eclipse, IntelliJ IDEA. Strong support for enterprise applications.	Swift, Java, JavaScript, Xcode, Android Studio, Visual Studio Code, Flutter. Needs platform-specific tools.

Recommendations

Based on the requirements for a scalable, web-based, multi-platform application, Linux is recommended as the most suitable operating platform for The Gaming Room. This recommendation is based on Linux's ability to support high-performance server environments, handle multiple users simultaneously, and integrate effectively with cloud-based services. These capabilities make it well suited for a distributed application where reliability, scalability, and cross-platform access are essential.

1. **Operating Platform**: A Linux-based server environment for The Gaming Room is the recommended operating platform for a web-based application. Linux is widely used for hosting due to its strong performance, security, and scalability. Developing the game as a web-based application will allow users to access *Draw It or Lose It* from multiple platforms (Windows, Mac, mobile devices) without requiring separate installations. This platform also supports open-source tools and frameworks, which can reduce development and operational costs. Linux servers are commonly used in cloud environments, making it easier to integrate with cloud services for storage and scaling. Linux's ability to maintain a high level of stability also supports the demands of a multi-user environment with simultaneous activity.

2. **Operating Systems Architectures**: The multi-user and multitasking architecture of Linux allows multiple players to interact with the application at the same time. The server handles the game logic and data while the clients utilizing web browsers can interact with the system. This architecture follows a client-server model. The server is responsible for processing requests and the client provides the user interface that allows user interaction. The separation that occurs in this type of architecture improves performance by keeping critical processes on the server side while still being able to make changes on the client side. Updates and maintenance can be performed without making changes to the server side, improving efficiency.

3. **Storage Management**: A cloud-based storage system or database is recommended because Linux systems integrate well with these databases, allowing data to be accessed from multiple locations. This cloud storage allows for scalability to support a higher demand for data as the application and number of users grows. Utilizing a database provides efficient organization and fast retrieval of data, which is important for the images and user data the game will require.

4. **Memory Management**: The efficient memory management techniques of Linux ensure access to the game by multiple users simultaneously. Linux manages memory efficiently by allocating resources based on system demand. Because memory is distributed based on system demand, the application is able to maintain high performance and process various workloads. This helps the stability of the application and prevents issues, such as lag when loading the images, from occurring.

5. **Distributed Systems and Networks**: The application would work best using a system where the server hosts the game and the clients use web-browsers over the internet, like in a client-server distributed system. The network plays a critical role in allowing communication between devices regardless of their operating system. Web protocol, such as HTTP and HTTPS, provide a standard way for data to be transmitted between the client and server. This network communication supports scalability by allowing addition users to connect without needing changes to devices. Accessibility is also significantly improved because users only need an internet connection to play the game.

6. **Security**: Linux has strong security features that are built in, such as user permissions and access controls. HTTPS encryption enhances this existing security. Measurements such as these ensure there is safe communication between clients and servers while also helping to protect user data. Additional security measures can include authentication systems to verify user identity and prevent unauthorized access. Encryption helps protect sensitive data transmission and access control ensures only authorized users can view or modify data. Further protections like firewalls, which can be used to protect a system connected to a network, are also important for server environments handling multiple user connections over the internet.